

Chapter 7 One-Pager — Opponent Modeling

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. A Nash-equilibrium strategy is unexploitable but blind: it plays identically against everyone and never punishes a weak opponent. Opponent modeling is the sensor that turns observed actions into an estimate of a specific opponent’s strategy, so the agent can deviate from equilibrium to exploit them — the first half of the thesis’s Behavioral Adaptation Framework (Contribution #1). The hard part: doing so without becoming exploitable yourself.

Approach. Bayesian belief-update loop (prior x likelihood -> posterior -> best response), built behind one shared interface so a single exact best response can be applied to three interchangeable models: (1) **type-based** — a belief over a fixed menu of opponent types; (2) **continuous** — free-form per-situation Dirichlet counts; (3) **consistent** — a sequence-form convex-optimization estimate (Ganzfried 2025). An **adaptive exploiter** runs the observe -> model -> best-respond -> act loop, with a Nash safety blend and an optional change-point detector for non-stationary opponents. Testbeds: Kuhn Poker and Leduc Hold’em, both exactly solvable, so every result is bracketed by exact analytical references.

Key results (measured).

- *Modeling is worth it.* Equilibrium play leaves **0.11-0.28 per hand** on the table against exploitable Kuhn opponents; the gap against a Nash opponent is ~ 0 (you cannot exploit an equilibrium).
- *When the model class fits, best response reaches the exact ceiling* — the type-based model stays within 3% of the best-response ceiling on every opponent in both games (and within two standard errors on nearly all).
- *A confident-but-underfit model makes you exploitable.* On Leduc the continuous model **loses to Nash (-0.175 vs a -0.083 ceiling)** — best-responding to a wrong estimate of an unexploitable opponent opens a leak in its own play. This is the exploitation-vs-safety tension in a single number.
- *Confident-and-wrong is a real risk.* Against an out-of-menu opponent, the type-based posterior commits hard to the nearest representable type; over 300 seeds a wrong type led past hand 100 in $\sim 13\%$ of runs (it always corrects eventually, and never falls once locked).
- *Non-stationarity is scenario-dependent.* Change-point forgetting rescues a harmful stale model (Kuhn: $-0.106 \rightarrow +0.211$) but underperforms simple adaptation when the new opponent is exploitable and the detector false-fires (Leduc). The reaction matters as much as the detection.
- *Consistency.* The consistent model recovers Kuhn strategies accurately (TV $\sim 0.004-0.021$) but its per-update convex solve is too costly for the online loop as built — an empirical answer to the chapter’s real-time-feasibility question, pointing to incremental solving / Chapter 8 approximations.

Thesis connection. Chapter 7 builds the **sensor** (the model); Chapter 8 builds the **actuator** (safe, KL-regularized exploitation). The continuous model’s Nash self-leak is the empirical case for that safety mechanism; the consistency theory is the principled backbone the framework extends.

Open questions. Real-time consistent modeling (incremental convex solve); principled non-stationarity (confidence-scaled forgetting, better change signals); out-of-menu opponents (open-world type discovery); one opponent to many (joint best response is not the combination of individual ones, and the convex guarantee breaks for $N > 2$).